

● PERSONAL

● PROMOTION

● BRAND

QA

Molestie end tiam tes cum vitae purus ex
pellentesque aur vehic la ligula litae auctor ou

READ MORE



QA AUTOMATION



Université Africaine de Sciences

MANAGEMENT DE LA QUALITE DU LOGICIEL

DR. AFFANE MOUNDOUNGA ANSELME R.

Objectifs du cours

- Comprendre l'importance de la qualité du logiciel et les différentes perspectives de la qualité.
- Connaître les modèles de qualité du logiciel couramment utilisés et les critères de qualité associés.
- Apprendre à utiliser les métriques de qualité du logiciel pour évaluer la qualité du code et des tests.

Objectifs du cours

- Maîtriser les techniques d'assurance qualité pour améliorer la qualité du logiciel.
- Comprendre les principes de la gestion de la qualité du logiciel, y compris la planification, le suivi et l'amélioration continue.
- Être capable d'appliquer les concepts et les pratiques de la qualité du logiciel dans des projets de développement logiciel réels.

Objectifs du cours

- Développer des compétences en communication et en travail d'équipe pour collaborer efficacement avec les parties prenantes dans le processus de développement logiciel.
- Se préparer à des certifications professionnelles en qualité du logiciel, telles que la certification ISTQB.

Introduction et définitions



Introduction

- L'information est aujourd'hui une ressource stratégique pour la plupart des entreprises, dans lesquelles de très nombreuses activités reposent sur l'exploitation d'applications informatiques.
- Pour ces entreprises, la fiabilité de leur système informatique et la qualité des logiciels utilisés sont donc cruciales.

Introduction

- Parallèlement, on constate que :
 - la part du logiciel est aujourd'hui prépondérante dans le coût total d'un système informatique,
 - la demande d'applications nouvelles, et de plus en plus complexes, ne cesse de croître,
 - les utilisateurs sont de plus en plus exigeants en termes de fiabilité et de sécurité.

Introduction

- A ce constat, il faut ajouter que :
 - de nombreux projets en cours de développement sont abandonnés pour non-conformité, retard trop important, dépassement de budget, etc.

Introduction

- A ce constat, il faut ajouter que :
 - les projets, qui aboutissent, accusent souvent un retard de plusieurs années et le coût du produit développé est souvent bien supérieur aux prévisions.

Introduction

- Il est donc nécessaire :
 - d'accroître la qualité du produit logiciel, tant dans les domaines de la satisfaction des utilisateurs, de la fiabilité, des performances, etc., que dans les domaines contractuels de respect des délais et des budgets.

Introduction

- Il est donc nécessaire :
 - d'accroître la productivité et de réduire les coûts, non seulement de production du logiciel, mais aussi de l'ensemble des activités mises en oeuvre au cours de son cycle de vie.

Génie logiciel

- Décrit l'ensemble de méthodes (spécification, conception, test...) , techniques et outils (Atelier de Génie logiciel) nécessaires à la production de logiciels de qualité industrielle.

Objectifs de génie logiciel

- Satisfaire les utilisateurs, en produisant des logiciels adaptés aux besoins ;
- les gestionnaires, en réduisant les coûts de maintenance et
- les chefs de projet, en rendant les logiciels productifs dans un délai raisonnable.

Risque de logiciel

- Diffusion massive en pilotage, système sécuritaire, industrie nucléaire (réservation de ticket, pilotage aéronautique, diagnostique médicale, opération financière ...)

Qualité du logiciel

- (Déf. Intuitive) La qualité est la conformité avec les besoins, l'adéquation avec l'usage attendu, le degré d'excellence ou , tout simplement, la valeur de quelque chose pour quelqu'un;

Qualité du logiciel

- (Déf. ISO) Ensemble des traits et des caractéristiques d'un produit logiciel portant sur son aptitude à satisfaire des besoins exprimés ou implicites;

Qualité du logiciel

- (Déf. IEEE) La qualité du logiciel correspond au degré selon lequel un logiciel possède une combinaison d'attributs désirés;
- Un logiciel est de qualité s'il possède les caractéristiques nécessaires pour répondre aux besoins des utilisateurs et s'il est conforme aux exigences spécifiées.

Qualité du logiciel : Déf. IEEE

- L'IEEE a développé un certain nombre de normes et de guides pour aider les organisations à améliorer la qualité de leurs logiciels;

Qualité du logiciel : Déf. IEEE

- Ces normes et guides couvrent un large éventail de sujets, tels que :
 - Le management de la qualité logicielle
 - L'assurance qualité logicielle
 - Les tests de logiciels
 - La maintenance logicielle

Qualité du logiciel

- (Déf. de Crosby) La qualité du logiciel correspond au degré selon lequel un client perçoit qu'un logiciel réponde aux multiples attentes;
- Selon Crosby, la qualité du logiciel est la conformité aux exigences spécifiées;

Qualité du logiciel : Déf. de Crosby

- **Qualité = Conformance aux exigences / Absence de défauts**
- Un logiciel est de qualité s'il répond parfaitement aux exigences des utilisateurs et ne contient aucun défaut.

Qualité du logiciel : Déf. de Crosby

- Crosby insiste sur l'importance de la prévention des défauts dès les premières étapes du développement logiciel.
- Il affirme que la qualité ne peut pas être contrôlée après coup, mais doit être conçue et intégrée dès le départ.

Points clés de la vision de Crosby sur la qualité

- **Conformité aux exigences** : La qualité n'est pas une notion abstraite de "luxe" ou de "supériorité", mais le respect strict des spécifications.
- **Rôle du Management** : Le management est responsable de fixer ces spécifications en fonction des besoins clients.

Points clés de la vision de Crosby sur la qualité

- **Zéro Défaut** (Zero Defects) : L'objectif est d'éliminer les erreurs dès la conception et le développement plutôt que de simplement les corriger après coup.
- **DRIFT** (Do It Right First Time) : Bien faire les choses dès la première fois pour éviter les coûts de non-qualité.

Qualité du logiciel

- (Déf. de Pressman) Conformité aux exigences explicites à la fois fonctionnelles et de performances, aux standards de développements explicitement documentés et aux caractéristiques implicites qui sont attendues de tous logiciels professionnellement développés

Qualité du logiciel : Déf. de Pressman

- **Qualité = Conformité aux exigences explicites et implicites**
- Un logiciel est de qualité s'il répond à toutes les exigences des utilisateurs, explicites et implicites.

Qualité du logiciel : Déf. de Pressman

- Les **exigences explicites** sont celles qui sont documentées.
 - Elles peuvent inclure les fonctionnalités du logiciel, les performances, la sécurité et la maintenabilité.

Qualité du logiciel : Déf. de Pressman

- Les **exigences implicites** sont celles qui ne sont pas documentées.
 - Elles peuvent inclure la convivialité, la facilité d'utilisation et la fiabilité du logiciel.

Différents points de vue

	Utilisateur	Vendeur	Gestionnaire de projet	Programmeur	Professeur
Facilité d'utilisation	X				
Compatibilité	X				
Multiple fonctions	X	X		X	
Haute performance	X	X			
0 défauts	X		X		
Développement rapide	X	X	X		
Faibles coûts de développement			X		
Code élégant				X	X

Logiciel

- Un ensemble des documents de gestion de projet,
- Une spécification décrivant la liste des fonctions à remplir par le logiciel, les facteurs qualité du logiciel (portabilité, évolutivité, robustesse, ...), les contraintes (performances temporelles et spatiales, ...) et les interfaces avec son environnement

Logiciel

- Une conception décrivant la découpe de la spécification en modules (ou objets), la description des interfaces entre ces modules (ou objets) et la description des algorithmes mis en place
- Un code source et un exécutable

Produit

- Programmes sources et machines, des procédures et des ensembles de données enregistrées.

Client et Fournisseur

- Le client commande un logiciel, le fournisseur le réalise.

Caractéristiques de logiciel

- **Produit unique**
 - Conçu et fabriqué une seule fois, reproduit
- **Inusable**
 - Défauts pas dus à l'usure mais proviennent de sa conception
- **Complexe**
 - Le logiciel est fabriqué pour soulager l'humain d'un problème complexe ; il est donc par nature complexe

Caractéristiques de logiciel

- **Invisible**

- Fabrication du logiciel = activité purement intellectuelle
- Difficulté de perception de la notion de qualité du logiciel

- **Technique non mature**

- Encore artisanal malgré les progrès

Plan de développement

- La description du logiciel à réaliser en **différents niveaux de produits** (programmes et documents)
- Les **moyens matériels et/ou logiciel** à mettre à disposition ou à réaliser (Méthodes, Techniques, Outils)

Plan de développement

- Le découpage du **cycle de vie** en phases, la définition des **tâches** à effectuer dans chaque phase et l'identification des **responsables** associés

Plan de développement

- Les supports de suivi de l'avancement (Planning et calendriers)
- Les moyens utilisés pour gérer le projet
- Les **points clés** avec ou sans intervention du client

Constat sur le développement de logiciel

- Le coût du développement du logiciel dépasse souvent celui du matériel,
- Les coûts dans le développement du logiciel :
 - 20% pour le codage et la mise au point individuelle
 - 30% pour la conception
 - 50% pour les tests d'intégration

Constat sur le développement de logiciel

- Durée de vie d'un logiciel de 7 à 15 ans
- Le coût de la maintenance évolutive et corrective constitue la part prépondérante du coût total
- Plus de la moitié des erreurs découvertes en phases de tests proviennent d'erreurs introduites dans les premières étapes

Constat sur le développement de logiciel

- La récupération d'une erreur est d'autant plus coûteuse que sa découverte est tardive

Etapes de développement de logiciel

- **Analyse (du problème)**
 - comprendre et recenser les besoins
 - Spécification (par exemple cahier des charges)

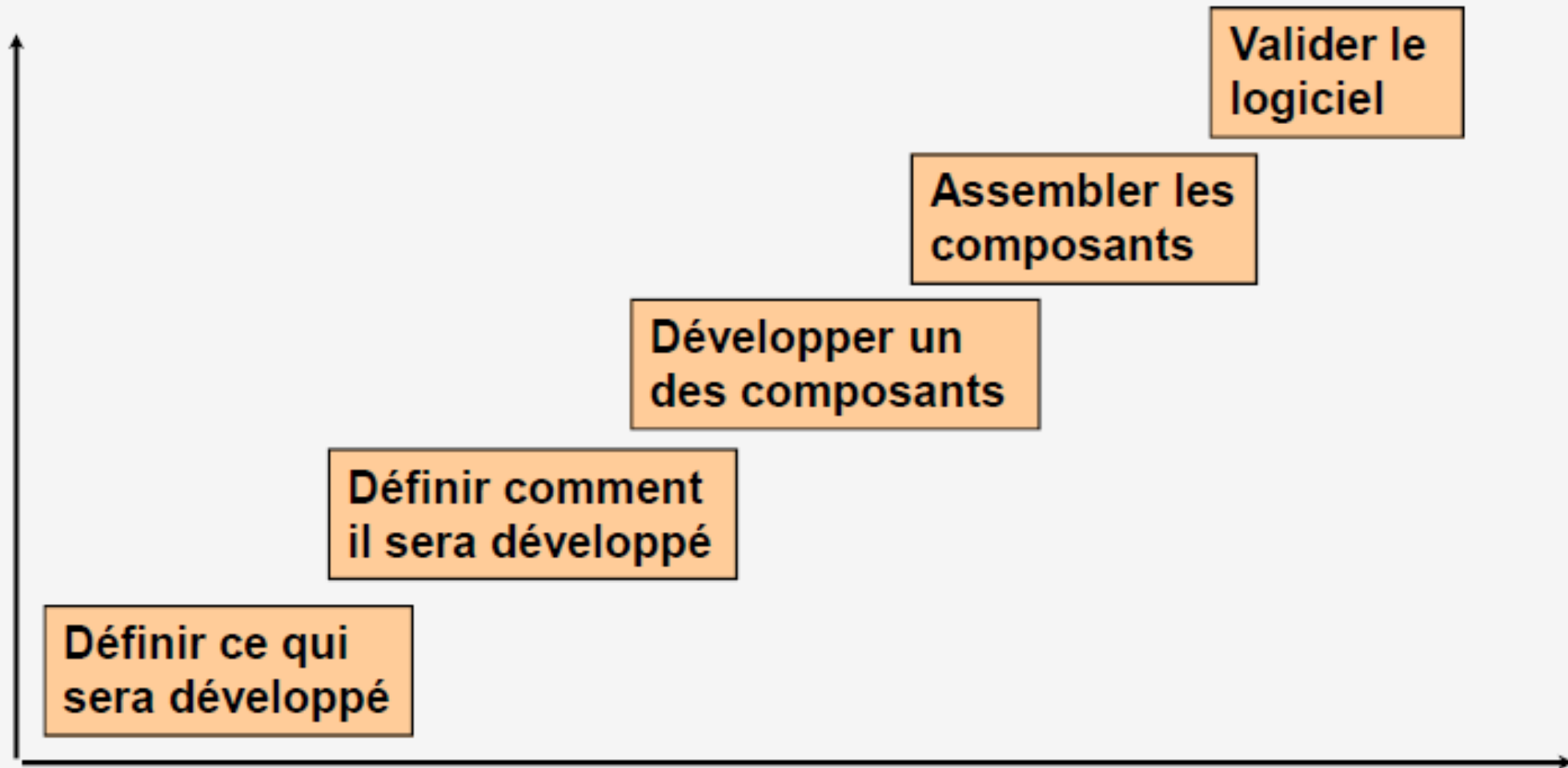
Etapes de développement de logiciel

- **Conception (du logiciel)**
 - **Préliminaire**
 - éclater le logiciel en sous-parties
 - définir les interfaces entre ces sous-parties
 - Définir l'architecture du logiciel
 - **Détaillée**
 - préciser l'architecture des sous-parties

Etapes de développement de logiciel

- **Implantation**
 - codage, intégration, tests

Activités de développement de logiciel



Activités de développement de logiciel

- L'organisation de ces activités et leur enchaînement définit le **cycle de développement du logiciel**

Importance de qualité de logiciel

- Le logiciel est une composante majeure des systèmes informatiques (environ 80% du coût) utilisés pour :
 - communication (ex. téléphone, syst. email)
 - santé (monitoring)
 - transport (ex. automobile, aéronautique)
 - échanges économiques (ex. commerce)
 - Entertainment

Importance de qualité de logiciel

- Les défauts du logiciel sont extrêmement coûteux en terme
 - d'argent
 - de réputation
 - de perte de vie

Facteurs de non qualité de logiciel

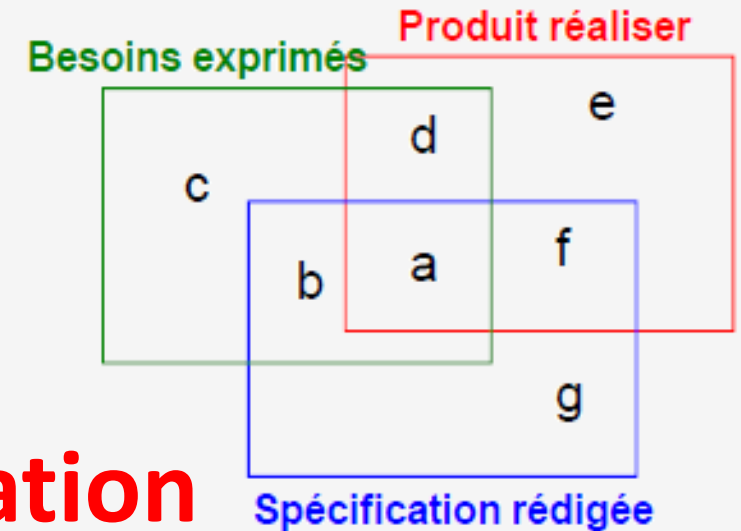
- **Mauvaise spécification :**
 - Vague, incomplète, instables...
- **Mauvaise estimation :**
 - Fausse, oublis, précisions insuffisantes...
- **Mauvaise répartition des tâches :**
 - Organisation inadaptée, contraintes omises
 - Ecart non détecté à temps

Facteurs de non qualité de logiciel

- **Mauvaise réalisation technique**
 - Codage, tests, documentation
- **Problèmes humains**
 - Mauvaise distribution des travaux
 - Conflits, rétention d'information
- **Manque d'expérience du métier de chef de projet**

Problème de conformité aux besoins

- a. Satisfaction
- b. Non-conformité
- c. Insatisfaction client
- d. Satisfaction hasardeuse
- e. Prestations inutiles hors spécification
- f. Spécifications et prestations inutiles
- g. Spécification inutiles





Critères de qualité logiciel

Critères de qualité logiciel

- ISO/IEC 9126 propose **6 caractéristiques** de qualité du produit logiciel :
 - **Capacité fonctionnelle** (Functionality)
 - **Fiabilité** (Reliability)
 - **Facilité d'utilisation** (Usability)
 - **Rendement** (Efficiency)
 - **Maintenabilité** (Maintainability)
 - **Portabilité** (Portability)

Capacité fonctionnelle

- **Définition**

- Ensemble d'attributs portant sur l'existence d'un **ensemble de fonctions et leurs propriétés.**
- Les fonctions sont celles qui satisfont aux besoins exprimés ou implicites;

Capacité fonctionnelle

- **Sous-caractéristiques**
 - **Aptitude** : présence et adéquation d'une série de fonctions pour des tâches données
 - **Exactitude** : fourniture de résultats ou d'effets justes ou convenus

Capacité fonctionnelle

- **Sous-caractéristiques**
 - **Interopérabilité** : capacité à interagir avec des systèmes donnés
 - **Sécurité** : aptitude à empêcher tout accès non autorisé (accidentel ou délibéré) aux programmes et données

Fiabilité

- **Définition**
 - Ensemble d'attributs portant sur l'**aptitude du logiciel** à maintenir son niveau de service dans des conditions précises et pendant une période déterminée

Fiabilité

- **Sous-caractéristiques**
 - **Maturité** : fréquence des défaillances dues à des défauts du logiciel
 - **Tolérance aux fautes** : aptitude à maintenir un niveau de service donné en cas de défaut du logiciel ou de violation de son interface

Fiabilité

- **Sous-caractéristiques**
 - **Possibilité de récupération** : capacité à rétablir son niveau de service et de restaurer les informations directement affectées en cas de défaillance ;
 - temps et effort nécessaire pour le faire

Facilité d'utilisation

- **Définition**

- Ensemble d'attributs portant sur l'**effort nécessaire** pour l'utilisation et l'évaluation individuelle de cette utilisation par un ensemble défini ou implicite d'utilisateurs

Facilité d'utilisation

- **Sous-caractéristiques**
 - **Facilité de compréhension** : effort que doit faire l'utilisateur pour reconnaître la logique et sa mise en œuvre
 - **Facilité d'apprentissage** : effort que doit faire l'utilisateur pour apprendre son application

Facilité d'utilisation

- **Sous-caractéristiques**
 - **Facilité d'exploitation** : effort que doit faire l'utilisateur pour exploiter et contrôler l'exploitation de son application

Rendement

- **Définition**
 - Ensemble d'attributs portant sur **le rapport** existant entre le niveau de service d'un logiciel et la quantité de ressources utilisées, dans des conditions déterminées

Rendement

- **Sous-caractéristiques**
 - **Comportement vis-à-vis du temps** : temps de réponses et de traitement ; débits lors de l'exécution de sa fonction
 - **Comportement vis-à-vis des ressources** : quantité de ressources utilisées ; durée de leur utilisation lorsqu'il exécute sa fonction

Maintenabilité

- **Définition**

- Ensemble d'attributs portant sur l'effort nécessaire pour faire des **modifications données**

Maintenabilité

- **Sous-caractéristiques**

- **Facilité d'analyse** : effort nécessaire pour diagnostiquer les déficiences et causes de défaillance ou pour identifier les parties à modifier
- **Facilité de modification** : effort nécessaire pour modifier, remédier aux défauts ou changer d'environnement

Maintenabilité

- **Sous-caractéristiques**
 - **Stabilité** : risque des effets inattendus des modifications
 - **Facilité de test** : effort nécessaire pour valider le logiciel modifié

Portabilité

- **Définition**

- Ensemble d'attributs portant sur l'aptitude du logiciel à être **transféré d'un environnement à l'autre**

Portabilité

- **Sous-caractéristiques**
 - **Facilité d'adaptation** : possibilité d'adaptation à différents environnements donnés sans que l'on ait recours à d'autres actions ou moyens que ceux prévus à cet effet pour le logiciel considéré

Portabilité

- **Sous-caractéristiques**
 - **Facilité d'installation** : effort nécessaire pour installer le logiciel dans un environnement donné
 - **Conformité aux règles de portabilité** : conformité aux normes et aux conventions ayant trait à la portabilité

Portabilité

- **Sous-caractéristiques**
 - **Interchangeabilité** : possibilité et effort d'utilisation du logiciel à la place d'un autre logiciel donné dans le même environnement

Compromis de qualité

- Les critères de qualité peuvent être **contradictaires**, pour chaque, le choix des compromis doit s'effectuer en fonction du **contexte**.
 - Facilité d'emploi **vs** maintenabilité
 - Fiabilité **vs** portabilité
 - Capacité **vs** rendement

Compromis de qualité

- La qualité s'obtient par la mise en place de **procédure** et des **méthodes** de phase de spécification
- La démarche qualité ne consiste pas à corriger les défauts mais à les **prévenir**

Schéma de compromis de qualité

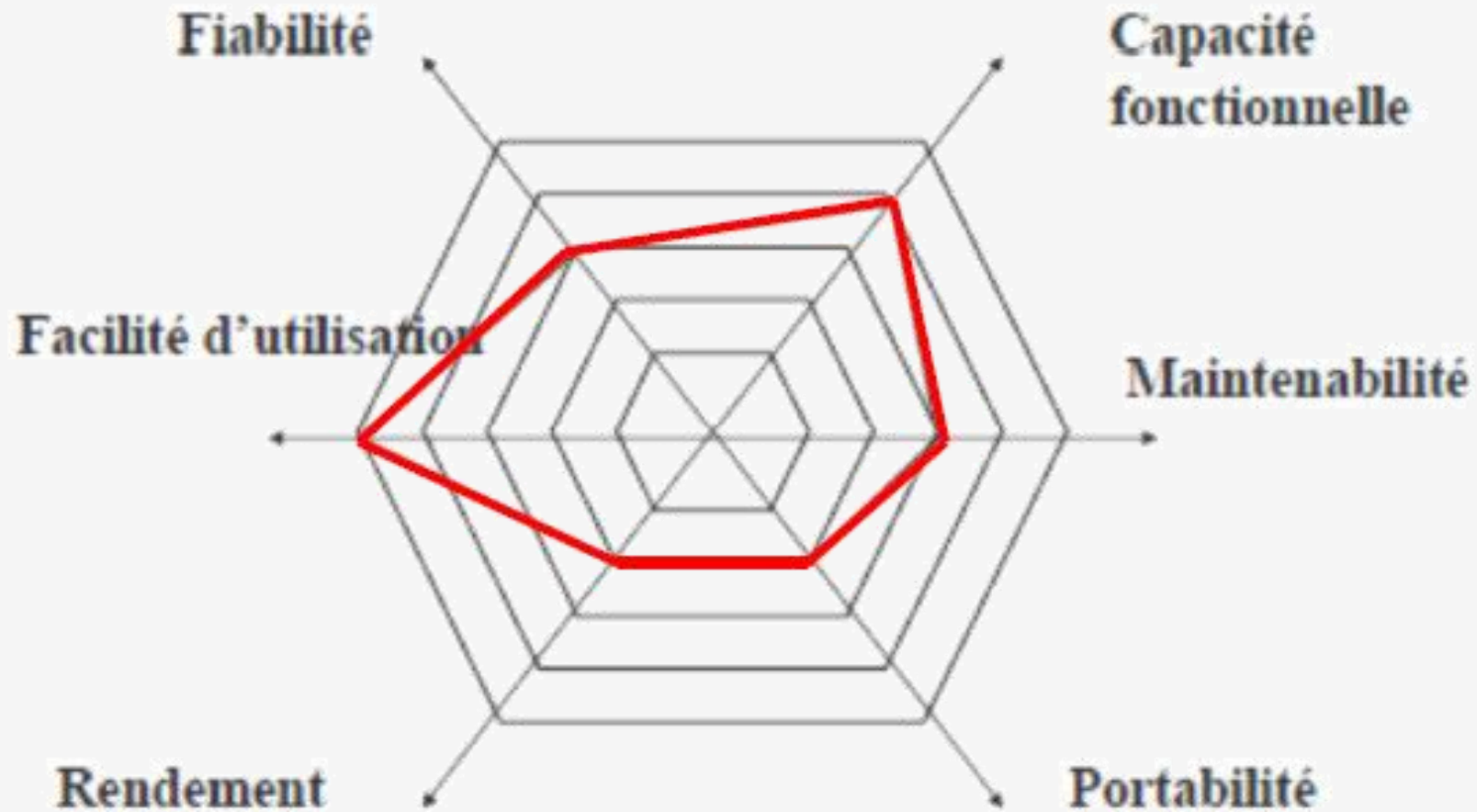


Schéma de compromis de qualité

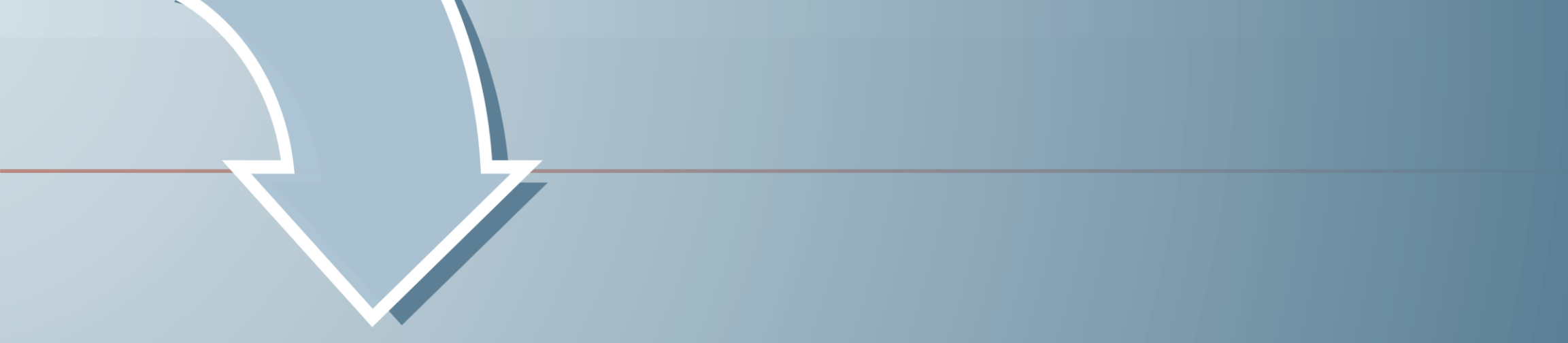
- Le schéma du compromis qualité du logiciel est un diagramme qui montre les relations entre les différentes dimensions de la qualité du logiciel.
- Ces dimensions sont généralement regroupées en deux catégories :

Schéma de compromis de qualité

- **Qualité interne** : Les caractéristiques du logiciel qui affectent son fonctionnement interne, telles que la fiabilité, la maintenabilité et le rendement.

Schéma de compromis de qualité

- **Qualité externe** : Les caractéristiques du logiciel qui affectent son utilisation par les utilisateurs, telles que la facilité d'utilisation, la portabilité et la capacité fonctionnelle.



Assurance qualité logiciel

Définition de l'AQL

- Un modèle planifié et systématique de toutes les actions nécessaires pour fournir une confiance adéquate qu'un article ou un produit est conforme à ses exigences techniques établies afin d'obtenir de la qualité requise.

Définition de l'AQL

- Un ensemble d'activités conçu pour évaluer le processus par lequel les produits sont développés ou fabriqués.

Définition de l'AQL

- Mise en oeuvre d'une approche préventive de la qualité : L'AQ consiste en un ensemble d'actions de prévention des défauts qui accompagnent le processus de développement des artefacts logiciels.
- Passe par l'élaboration d'un MANUEL QUALITE

Objectifs de l'AQL

- Assurer un niveau de confiance acceptable que le logiciel sera conforme aux exigences fonctionnelles techniques.
- Assurer un niveau de confiance acceptable que le logiciel sera conforme aux exigences de gestion concernant l'échéancier et le budget.

Objectifs de l'AQL

- Initier et gérer des activités visant à l'amélioration et la plus grande efficacité des activités de développement et d'assurance de qualité logicielle.

Objectifs de l'AQL

- Initier et gérer des activités visant à l'amélioration et à l'augmentation de l'efficacité des activités de maintenance et d'assurance de qualité logicielle

Contrôle de la qualité

- Ensemble des processus et méthodes pour contrôler le travail et observer si les exigences sont satisfaites.
- La sélection des caractéristiques d'évaluation ainsi que la spécification des techniques d'évaluation utilisée

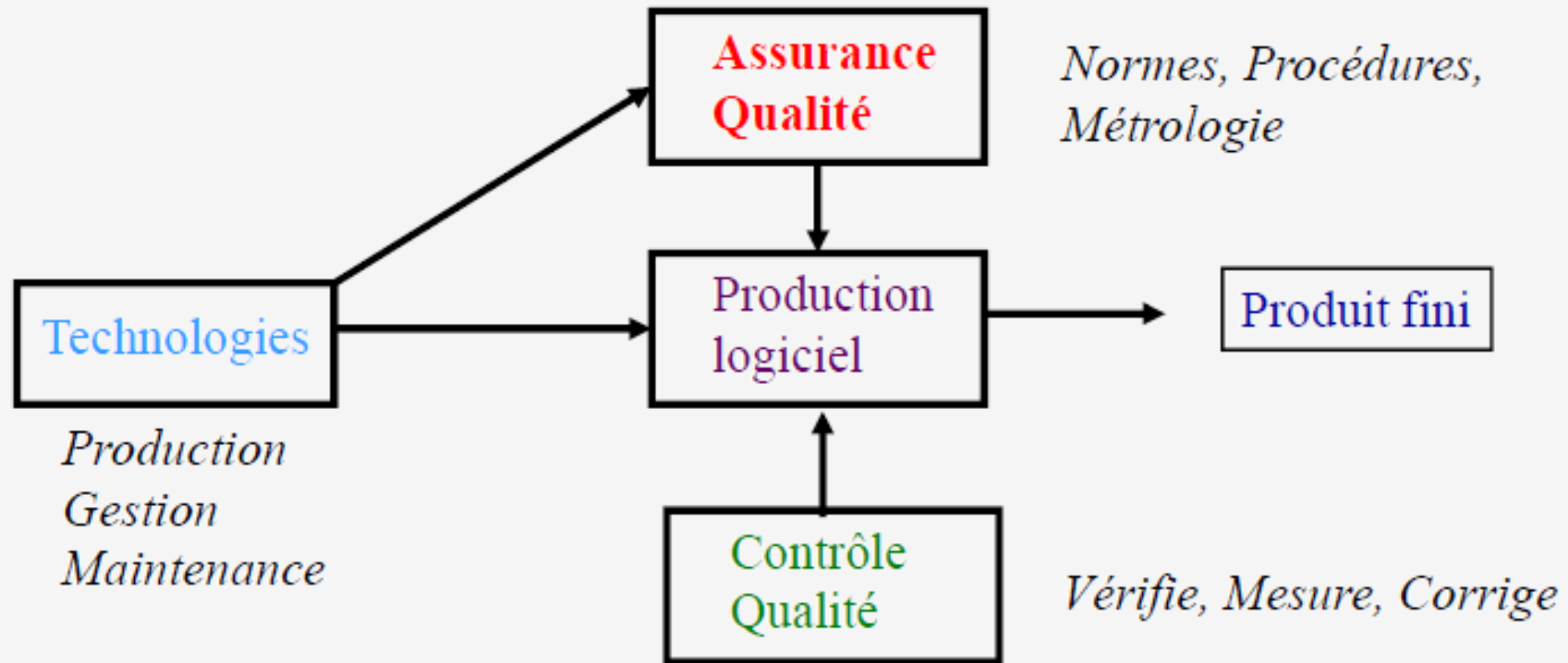
Contrôle de la qualité

- Les revues et la suppression des défauts avant la livraison du produit.
- La validation : effectuée à la fin du processus de développement pour assurer la conformité aux exigences du produit
 - répondre à la question "sommes nous en train de faire le bon produit?"

Contrôle de la qualité

- La vérification : effectuée à la fin d'une phase pour s'assurer que des besoins établis pendant la phase précédente ont été répondus
 - répond à la question "est ce que nous faisons le produit correctement"

Contrôle de la qualité



Directives complémentaires

- Les directives d'identification, d'implantation et d'analyse des métriques nécessaires au processus d'évaluation du produit final,
- Les directives de définition des indicateurs qui permettent des évaluations partielles pendant le cycle de développement

Directives complémentaires

- **But :**

- Les informations générales sur les indicateurs de qualité des logiciels ;
- Les critères de sélection de ces indicateurs
- Les directions pour l'évaluation des données de mesurage

Directives complémentaires

- **But :**

- Les directions pour l'amélioration du processus de mesurage
- Les exemples de types de graphes d'indicateurs

Manuel qualité

- Document décrivant les dispositions générales prises par l'entreprise pour obtenir la qualité de ses produits ou de ses services
- **Rôle** : Usage interne et externe

Manuel qualité

- **Organisé en 6 parties :**
 - Organisation de l'entreprise
 - Activités de production et de contrôle technique
 - Activité de gestion
 - Activité de contrôle de la qualité
 - Plan type du PLAN QUALITE
 - Lignes directrices permettant d'établir le plan qualité

Utilisation du manuel qualité

- Communiquer la politique, les procédures et les exigences
- Mettre en oeuvre un système effectif;
- Fournir une maîtrise améliorée des pratiques et faciliter les activités relatives à l'assurance;

Utilisation du manuel qualité

- Fournir les bases documentaires pour auditer les systèmes qualité;
- Assurer la continuité du système qualité et de ses exigences en cas de modification des circonstances;

Utilisation du manuel qualité

- Fournir les bases documentaires pour auditer les systèmes qualité;
- Assurer la continuité du système qualité et de ses exigences en cas de modification des circonstances;

Utilisation du manuel qualité

- Former le personnel aux exigences de système qualité et aux méthodes relatives à la conformité
- Présenter leur système qualité pour des usages externes, tel que la démonstration de leur conformité aux normes

Utilisation du manuel qualité

- Démontrer la conformité de leurs systèmes qualité avec les normes relatives à la qualité dans des situations contractuelles.

Plan qualité logiciel

- Démontrer la conformité de leurs systèmes qualité avec les normes relatives à la qualité dans des situations contractuelles.

Plan qualité logiciel

- Document décrivant les dispositions spécifiques prises par une entreprise pour obtenir la qualité du produit ou du service considéré

Plan qualité logiciel

- Il comprend les techniques et les activités à caractère opérationnel qui ont pour but à la fois de piloter un processus et d'éliminer les causes de fonctionnement **non satisfaisant à toutes les phases de la boucle de la qualité** en vue d'atteindre la meilleure efficacité économique

Plan qualité logiciel

- IL incluse : le **produit** auquel il est prévu d'être appliqué, les **objectifs** relatifs à la qualité du produit (exprimer ces objectifs en termes mesurables chaque fois que cela est possible), les **exclusions** spécifiques ainsi que la **période de validité**

Plan type de qualité logiciel

- 1) But, Domaine d'application et responsabilité:
- 2) Documents applicables et documents de références
- 3) Terminologie
- 4) Organisation

Plan type de qualité logiciel

- 5) Démarche de développement
- 6) Documentation
- 7) Gestion des configurations
- 8) Gestion des modifications
- 9) Méthodes, outils et règles

Plan type de qualité logiciel

- 10) Contrôle des fournisseurs
- 11) Reproduction, protection, livraison
- 12) Suivi de l'application du plan qualité (plan de contrôle)

Principes généraux de l'AQL

- **La définition des exigences de qualité :**
interpréter le contexte
- **La préparation des processus d'évaluation :**
Planifier les activités de contrôle qualité
- **L'exécution de la procédure d'évaluation :**
réaliser l'évaluation du logiciel avant et après l'exécution

La définition des exigences de qualité

- Avoir des exigences et spécifications explicites
- Avoir un processus de développement logiciel avec:
 - Analyse des exigences,
 - Tests d'acceptabilité,
 - Feedback fréquent des usagers

La préparation de processus d'évaluation.

- À ce niveau, l'objectif est d'initier l'évaluation et de mettre au point ses bases. Ceci est fait en trois sous-étapes :
 - **La sélection des métriques de qualité.** Ces dernières doivent correspondre aux caractéristiques énumérées plus haut.

La préparation de processus d'évaluation.

- À ce niveau, l'objectif est d'initier l'évaluation et de mettre au point ses bases. Ceci est fait en trois sous-étapes :
 - **La définition des taux de satisfaction.** Les échelles de valeurs doivent être divisées en portions correspondant aux niveaux de satisfaction des exigences.

La préparation de processus d'évaluation.

- **La définition des critères d'appréciation.** Ceci inclut la préparation de la procédure de compilation des résultats par caractéristique.
- Il est possible aussi de prendre en compte dans cette procédure des aspects de gestion tels que le temps ou les coûts.

L'exécution de la procédure d'évaluation

- Il s'agit de donner un cadre rigoureux pour :
 - Guider le développement du logiciel, de sa conception à sa livraison.
- Contrôler les coûts, évaluer les risques et respecter les délais.

L'exécution de la procédure d'évaluation

- Quatre méthodes complémentaires:
 - **Méthodes formelles** : Vérifier mathématiquement des propriétés spécifiées
 - **Métriques** : Mesurer un ensemble connu de propriétés liées à la qualité

L'exécution de la procédure d'évaluation

- **Quatre méthodes complémentaires:**
 - **Revue et Inspections** : Relecture et examen par humain des exigences, design, code, ... basés sur des checklists
 - **Tests** : Données explicites pour exécuter le logiciel et vérifier si les résultats correspondent aux attentes

Méthodes formelles

- **Preuves de**
 - **Correction** : L'expression formelle des spécifications
 - **Terminaison de l'exécution** : La production de la preuve d'une conception et d'une implémentation sans erreurs
 - **Propriétés spécifiques** : L'explicitation précise des propriétés d'architecture de développement

Méthodes formelles

- Assistants de preuve (theorem prover)
 - systèmes : Coq, PVS, Isabelle / HOL, ...
 - Encore difficiles d'usage pour des non spécialistes

Métriques

- Mesures des processus (les séries d'activités reliées au développement du logiciel), des produits (les objets produits, livrables ou documents qui résultent d'une activité d'un processus), des ressources (les entités exigées par une activité d'un processus)

Métriques

- **Mesure des caractéristiques « internes »**
 - mesures objectives : taille, complexité du flot de contrôle, cohésion modulaire / couplage entre modules, ...

Métriques

- **Mesure des caractéristiques « externes »**
 - évaluations stochastiques (statistiques) : délai moyen de réponse à une requête, nombre de requêtes simultanées sans « écrouler » un serveur, ...
- Ce sont des mesures postérieurs : arrivent parfois « trop tard »

Métriques

- **Métriques de McCabe**
 - Analyse du graphe de contrôle
 - Mesure de la complexité structurelle ;
 - Nombre cyclomatique : Représente le nombre de chemins indépendants

Métriques

- Métriques de McCabe
 - Nombre cyclomatique :

$$C = a - n + i + s$$

n : nb de noeuds (blocs d'instructions séquentielles)

a : nb d'arcs (branches de programme)

i : nb de noeud d'entrée

s : nb de noeud de sortie

Métriques

- Plus le nombre cyclomatique est grand, plus il y aura de chemins d'exécution dans la fonction, et plus elle sera difficile à comprendre.

Métriques

- Métriques de McCabe
 - **Calcul direct** : McCabe a introduit une nouvelle manière de calculer la complexité structurelle
 - **$C = P_i + 1$** , avec **P_i** le nombre de décision du code

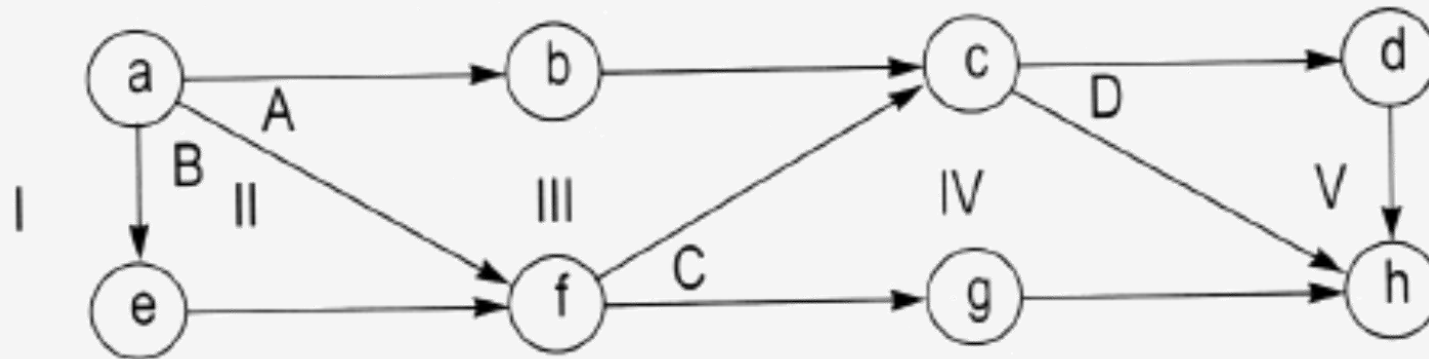
Métriques

- Métriques de McCabe

- Une instruction **IF** compte pour 1 décision
- Une boucle **FOR** ou **WHILE** compte pour **1** décision
- Une instruction **CASE** ou tout autre embranchement multiple compte pour **1** décision de moins que le nombre d'alternatives

Métriques

- Le nombre de faces du graphe donne la complexité structurelle du code
 - Exemple: $C = 4 + 1 = 5$



Métriques

- **Métriques de McCabe**

- Le nombre cyclomatique donne une évaluation du **nombre des chemins indépendants** dans le graphe et donc une indication sur le **nombre de tests nécessaires**

Métriques

- **Métriques de McCabe**

- Cette métrique indique la borne supérieure du nombre de tests à effectuer pour que tous les arcs soient couverts au moins une fois.
- Dans la pratique il semble que la limite supérieure du nombre cyclomatique soit de 30 environ.

Métriques de McCabe

- La complexité cyclomatique d'une méthode augmente proportionnellement au nombre de points de décision.
- Une méthode avec une haute complexité cyclomatique est plus difficile à comprendre et à maintenir.

Métriques de McCabe

- Une complexité cyclomatique trop élevée (supérieure à 30) indique qu'il faut refactoriser la méthode.

Métriques de McCabe

- La complexité cyclomatique est liée à la notion de "**code coverage**", c'est à dire la couverture du code par les tests.
- La **couverture de code** est une mesure qui indique quelle proportion du code source d'un programme a été exécutée par un ensemble de tests. Elle est généralement exprimée en pourcentage.

Métriques de McCabe

- Dans l'idéal, une méthode devrait avoir un nombre de **tests unitaires** égal à sa **complexité cyclomatique** pour avoir un "code coverage" de 100%.
- Cela signifie que chaque chemin de la méthode a été testé.

Métriques de McCabe – Exercice 1

- Calculer le nombre cyclomatique du programme suivant:

```
void sort(int *px, int n) {  
  
    int i, j, temp;  
  
    for (i = 2; i < n; i++) {  
        for (j = 1; j < i; j++) {  
            if (px[i] < px[j]) {  
                temp = px[i];  
                px[i] = px[j];  
                px[j] = temp;  
            }  
        }  
    }  
}
```

Métriques de McCabe – Exercice 2

- soit le programme « recherche dichotomique »
en langage C :

*Calculer le nombre
cyclomatique de cette
procédure.*

```
void recherche_dico (elem cle, elem t[], int taille, boolean &trouv, int &A)
{ int d, g, m;
  g=0; d=taille -1;
  A=(d+g) /2;
  if (t[A]= =cle) trouv=true;
  else trouv=false;
  while (g <=d && !trouv)
  { m= (d+g) /2;
    if (t[m]= =cle)
    {      trouv=true;
      A=m;
    }
    else if (t[m]> cle) g=m+1;
    else d=m-1;
  }
}
```

Métriques de McCabe – Exercice 3

- Consider the following program module :

```
int module1 (int x, int y) {  
    while (x != y) {  
        if (x > y)  
            x = x - y;  
        else  
            y = y - x;  
    }  
    return x;  
}
```

- Quelle est la complexité du module 1?

Métriques de McCabe – Exercice 3

```
int module1 (int x, int y) {  
    while (x != y) {  
        if (x > y)  
            x = x - y;  
        else  
            y = y - x;  
        }  
    return x;  
}
```

Explanation :

Noeuds de Condition:

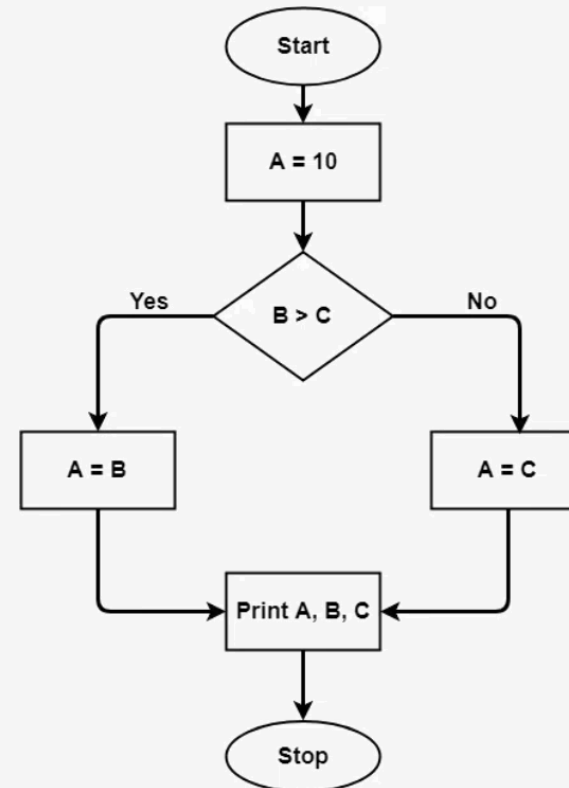
1.While

2. if

Cyclomatic Complexity=2+1=3

Exemple de flow de contrôle

```
A = 10  
IF B > C THEN  
  A = B  
ELSE  
  A = C  
ENDIF  
Print A  
Print B  
Print C
```



Métriques de Halstead

- Complexité liée à la distribution des variables et instructions.
- Fondées empiriquement et toujours considérées comme valides, contrairement à ses formules complexes de prédiction
- Métrique textuelle pour évaluer la taille d'un programme.

Métriques de Halstead

- Alternative au calcul du nombre de lignes de code source.
- **Entités de base** : Opérandes (jetons qui contiennent une valeur) et Operateurs (tout le reste :virgules, parenthèses, operateurs arithmétiques...)

Métriques de Halstead

- La base des mesures est fournie par le vocabulaire utilisé.
- On évalue le nombre d'opérateurs et d'opérandes.

Métriques de Halstead

- **n1** = nombre d'opérateurs uniques
- **n2** = nombre d'opérandes uniques (termes, constantes, variables)
- **N1** = nombre total d'apparition de ces opérateurs
- **N2** = nombre total d'apparition de ces opérandes.

Métriques de Halstead - Exemple

- $a := a + 1;$
 - 3 opérateurs $+$ $:=$ $;$
 - 2 opérands a et 1

Métriques de Halstead

- Mise en oeuvre :
 - Une fois que le code a été écrit, cette mesure peut être appliquée pour prédire la difficulté d'un programme et d'autres quantités, en employant les **équations de Halstead**

Métriques de Halstead

- Vocabulaire du programme

- $I = n1 + n2$ (E1)

- Taille observée du programme

- $L = N1 + N2$ (E2)

- Taille estimée du programme

- $Le = n1(\text{Log2 } n1) + n2 (\text{Log2 } n2)$ (E3)

Métriques de Halstead

- **Volume du programme** : Estimation du nombre de bits nécessaires pour coder le programme mesuré
 - **$V = L \text{Log}_2 (n_1 + n_2)$** **(E4)**

Métriques de Halstead

- Difficulté du programme : propension d'erreurs du programme
 - $D = (n1/2) (N2/n2)$

Métriques de Halstead

- Niveau du programme
 - $L1 = 1/D$
- Effort : Mesure de l'effort intellectuel nécessaire à l'implémentation d'un algorithme
 - $E = V/L1$

Métriques de Halstead

- Nombre d'erreurs : une estimation du nombre d'erreurs dans le programme
 - $B = E^{(2/3)} / 3000$

Métriques de Halstead

- Temps (de programmation) : Temps nécessaires pour implémenter un algorithme, où S est le nombre de Stroud (=18 pour Halstead)
 - $T = E/S$
 - $T = E/D$

Métriques de Halstead – Exercice 1

- Calculer les mesures de Halstead pour le programme suivant :

```
int z = 0;
while (x > 0) {
    z = z + y;
    x = x - 1;
}
printf("%d", z);
```

Métriques de Halstead - Exercice 2

- Calculer les mesures de Halstead pour le programme suivant :

```
void sort(int *px, int n) {  
  
    int i, j, temp;  
  
    for (i = 2; i < n; i++) {  
        for (j = 1; j < i; j++) {  
            if (px[i] < px[j]) {  
                temp = px[i];  
                px[i] = px[j];  
                px[j] = temp;  
            }  
        }  
    }  
}
```

Métriques de Halstead - Exercice 3

```
int sort (int x[ ], int n)

{
int i, j, save, im1;
/*This function sorts array x in ascending
order */
If (n< 2) return 1;
for (i=2; i< =n; i++)
{
im1=i-1;
for (j=1; j< =im1; j++)
if (x[i] < x[j])
{
Save = x[i];
x[i] = x[j];
x[j] = save;
}
}
return 0;
}
```

Métriques de Halstead - Exercice 3

```
int sort (int x[ ], int n)

{
int i, j, save, im1;
/*This function sorts array x in ascending
order */
If (n< 2) return 1;
for (i=2; i< =n; i++)
{
im1=i-1;
for (j=1; j< =im1; j++)
if (x[i] < x[j])
{
Save = x[i];
x[i] = x[j];
x[j] = save;
}
}
return 0;
}
```

Opérateurs	Occurrences	Operands	Occurrences
int	4	sort	1
()	5	x	7
,	4	n	3
[]	7	i	8
if	2	j	7
<	2	save	3
;	11	im1	3
for	2	2	2
=	6	1	3
-	1	0	1
<=	2	-	-
++	2	-	-
return	2	-	-
{ }	3	-	-
n1=14	N1=53	n2=10	N2=38

Métriques de Halstead - Exercice 3

Program Length (N) = 91
Vocabulary (n) = 24
Volume (V) = 417.23 bits
Estimated Program Length (N[^]) = 86.51
Potential Volume (V) = 11.6*
Program Level (L) = 0.027
Difficulty (D) = 37.03
Estimated Program Level (L[^]) = 0.038
Effort (T) = 610 seconds

Operators	Occurrences	Operands	Occurrences
int	4	sort	1
()	5	x	7
,	4	n	3
[]	7	i	8
if	2	j	7
<	2	save	3
;	11	im1	3
for	2	2	2
=	6	1	3
-	1	0	1
<=	2	-	-
++	2	-	-
return	2	-	-
}	3	-	-
n1=14	N1=53	n2=10	N2=38

Métriques de Henry-Kafura

- Mesurer la complexité des modules d'un programme en fonction des liens qu'ils entretiennent

Métriques de Henry-Kafura

- On utilise pour chaque **module i** :
 - Le **nombre de flux d'information entrant** noté **in_i**
 - Le **nombre de flux d'information sortant** noté **out_i**

Métriques de Henry-Kafura

- On utilise pour chaque **module i** :
 - Le **poids du module** noté **poids_i** calculé en fonction de son nombre de LOC et de sa complexité
 - **$HK_i = poids_i * (out_i * in_i)^2$**
- La mesure totale **HK** correspond à la somme des **HK_i**

Métriques de Henry-Kafura – Exercice

- A partir des **in_i** et **out_i** ci-dessous, calculer les métriques **HK** en supposant que le poids de chaque module vaut 1.

Module	a	b	c	d	e	f	g	h
In _i	4	3	1	5	2	5	6	1
Out _i	3	3	4	3	4	4	2	6
HK _i	?	?	?	?	?	?	?	?
HK=Σ(HK _i)	?							

Revue et Inspections

- Examen détaillé d'une spécification, d'une conception ou d'une implémentation par une personne ou un groupe de personnes, afin de détecter des fautes, des violations de normes de développement ou d'autres problèmes.

Revue et Inspections

- **Examen systématique** de certains points pour la **recherche** de potentiels défauts et la **vérification** de l'application de certaines règles menée par un spécialiste du domaine inspecté

Type de Revues et Inspections

- **Auto-correction (desk-checking)**
 - relecture **personnelle**
 - **bilan** : efficacité quasi nulle pour les documents amont, faible pour le code

Type de Revues et Inspections

- **Lecture croisée (author-reader cycle)**
 - un **collègue** recherche des ambiguïtés, oublis, imprécisions
 - **bilan** : efficacité faible pour les documents amont, plus adapté pour la relecture du code

Type de Revues et Inspections

- **Revue (walkthrough)**
 - **discussion** informelle au sein d'un groupe
 - un lecteur **résume** paragraphe par paragraphe
 - **bilan** : contribution moyenne à l'assurance qualité (évaluation très liée à la prestation du lecteur)

Type de Revues et Inspections

- **Revue structurée**

- constitution pendant le **débat** d'une liste de défauts, utilisation d'une liste de défauts typiques (**checklist**)
- direction des débats par un secrétaire
- **bilan** : bonne contribution à l'assurance qualité

Type de Revues et Inspections

- **Inspection**
 - cadre de relecture plus **formel**
 - recherche des défauts **avant** les débats
 - **suivi** des décisions et corrections
 - **bilan** : excellente contribution à l'assurance qualité

Organisation des Revues et Inspections

- **1. Préparation**
- **2. Cycle de réunions**
- **3. Suivi**

Organisation des Revues et Inspections

- **1. Préparation**
 - Recherche des défauts
 - Rédaction d'un rapport de défauts basé sur des fiches type

Organisation des Revues et Inspections

- **1. Préparation**
 - Recherche des défauts
 - Rédaction d'un rapport de défauts basé sur des fiches type
- **2. Cycle de réunions**
 - Examen de défauts
 - Prise de décision

Organisation des Revues et Inspections

- **3. Suivi**
 - Vérification des corrections ou nouvelle inspection

Défauts typiques

- **Référence aux données :**
 - Variables non initialisées
 - Indices de tableaux hors bornes
 - Accès à des structures/records à champs variables ou à des unions
 - Confusion entre donnée et pointeur vers la donnée

Défauts typiques

- **Référence aux données :**
 - Déréférence de pointeurs nuls
 - Pointeurs sur des données désallouées ou pas encore allouées
 - Pointeurs sur des données devenues inutiles mais non libérées

Défauts typiques

- **Calculs :**
 - **Conversions** de type (implicites et explicites)
 - **Underflow/overflow** (dépassement de capacité du type)
 - **Division par zéro**
 - **Précédence** des opérateurs

Défauts typiques

- **Comparaison :**
 - incohérence des types :
mélanges d'entiers et de booléens
 - inclusion ou non des bornes incorrecte :
< au lieu de <=, >= au lieu de >, ...

Défauts typiques

- **Comparaison :**
 - inversion du test :
== au lieu de !=, > au lieu de <, ...
 - confusion en égalité (==) et affectation (=) :
if (x = y) {...} au lieu de if (x == y) {...}

Défauts typiques

- **Comparaison :**

- confusion entre opérateurs binaires (bits) et logiques :

et (&, &&, and), ou (|, ||, or), négation (~, !, not)

- négation incorrecte d'une condition logique

`!(x ==1 && (y < 2 || !z))` équiv. `x !=1 || (y >= 2 && z)`

Défauts typiques

- **Comparaison :**
 - précedence des opérateurs booléens
 $x \ || \ y \ \&\& \ z \text{ équiv. } x \ || \ (y \ \&\& \ z)$

Défauts typiques

- **Contrôle :**
 - **Switch** : ensemble de case incomplet, cas default manquant, break oublié
 - **Rattachement du else au if**
 - **Terminaison du programme** : boucles et récursions sans fin

Défauts typiques

- **Contrôle :**
 - **Boucles** : conditions initiales (indices, ...) incorrectes, itérations en plus ou en moins, incohérences après une sortie de boucle anticipée, incohérences après une sortie de boucles emboîtées

Défauts typiques

- **Contrôle :**
 - **Procédures et fonctions** : incohérences après une sortie anticipée
 - **Exceptions non rattrapées**

chek-liste

- Commentaires
- Structure du code
- Références aux données
- Calculs
- Forme des décisions
- Définition de constantes

chek-liste

- Comparaisons
 - Contrôle
 - Taille des modules
-
- Le plan qualité prévoit en général l'utilisation de **règles de programmation** qui doivent également être vérifiées.

Tests

- Le test est l'exécution ou l'évaluation d'un système ou d'un composant par des moyens automatiques ou manuels, pour vérifier qu'il répond à ses spécifications ou identifier les différences entre les résultats attendus et les résultats obtenus

Tests

- Vérification (conformité aux spécifications)
 - tests unitaires
 - tests d'intégration

Tests

- Qualification
 - validation par rapport aux contraintes non fonctionnelles
 - tests de performance
 - tests de capacité de charge

Tests

- Qualification
 - validation par rapport aux besoins
 - bêta-test (chez l'utilisateur final)

Défauts typiques – Exercice 1

- Trouver 4 erreurs

```
int factorielle(int n)
{
    int f;
    while (n >= 0) {
        n = n-1;
        f = f*n;
    }
    return n;
}
```

Défauts typiques – Exercice 1

Trouver 4 erreurs

```
int factorielle(int n)
{
    int f;
    while (n >= 0) {
        n = n-1;
        f = f*n;
    }
    return n;
}
```

- variable f non initialisée (à 1)
- tour de boucle en trop (n = 0)
- opération sur un mauvais indice (n-1 et non n)
- retour d'une mauvaise variable (ici n)

Défauts typiques – Exercice 1

```
int factorielle(int n)
{
    int f = 1;
    while (n > 0) {
        f = f*n;
        n = n-1;
    }
    return f;
}
```

- variable f initialisée (à 1)
- pas de tour de boucle en trop
- opération sur un bon indice (n)
- retour de la variable correcte (f)